# Solution: Exercises for the Lecture Advanced Methods of AI

*Asking, Defining, Constraining, Deriving, Generating*

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Last week you *represented* the pizza world in first-order logic and in Python. Tonight a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression, a **description**, written in the Entity Query Language (EQL). The five acts of the evening are five things the robot does with one:

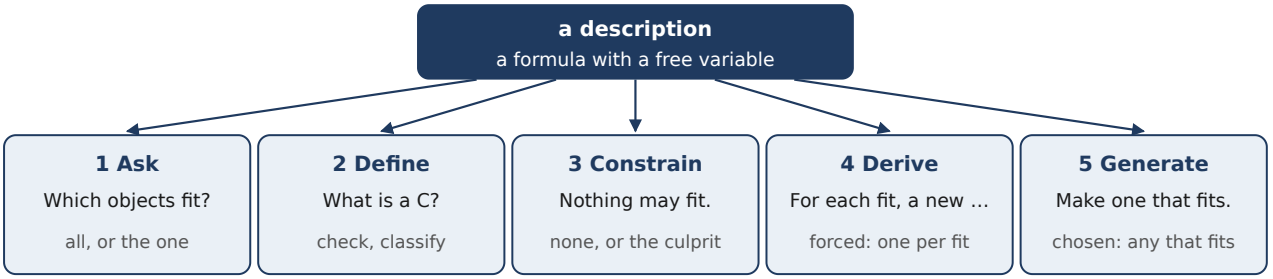



**Figure 1.** One description, five uses. The expression stays the same; what changes is what is done with it. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it.

FOR THE TUTOR

# The 90 minutes

| Time | Part | Students | Tutor, and the sentence that leads on |
|---|---|---|---|
| 0–8 | Slides | open the notebook and start its setup cell, then listen | slides 1–8. *“Let us look at the world the robot knows.”* |
| 8–12 | Prologue | warm-up on paper, in pairs | collect domain and extensions (2 min). *“The chef has a first question.”* |
| 12–36 | Act 1 Asking | 1(a) on paper; 1(b) predict and run; the vegetarian query; 1(c); 1(d) predict; 1(e) | stop after 1(b) for vacuous truth and the closed world (3 min); at 1(d) ask: what should the robot do with 0, and with 2? *“You asked which pizzas are vegetarian. What does the word mean?”* |
| 36–54 | Act 2 Defining | 2(a) on paper; 2(b) on paper, predict and run; the think question; 2(c) predict | after 2(a): no declaration can make an object a pizza. Draw ↔ on the board, ← classify, → conclude. *“Some descriptions must never fit anything.”* |
| 54–66 | Act 3 Constraining | 3(a) table, `allergens`, `IsSafeFor`; 3(b) on paper, then `dangerous` | draw ¬∀ ≡ ∃¬ on the board; one pair shows its red line. *“The labels were facts about pizzas. Now the kitchen writes new things.”* |
| 66–78 | Act 4 Deriving | 4(a); 4(b) predict and run | 4(b) is the climax: three pairs answer one source each. *“So far, every new object was forced. Dana orders one she chooses.”* |
| 78–85 | Act 5 Generating | 5(a) predict and run; 5(b) on paper | collect 5(b) on the board: several answers are right. *“Let us put the evening together.”* |
| 85–90 | Epilogue | 5(c) summary table | show Figure 14; the take-home message. |

### If time runs short

Skip the think question of Act 3 and assign 5(b) for home. Pairs that are fast continue with the think questions; nobody waits. Every notebook cell runs as it is, so a pair that is stuck on a `TODO` goes on and compares with the solution later.

PROLOGUE · 4 MINUTES

# The world the robot knows

*Thursday evening, 19:30*

> *A small pizzeria in Bremen. A service robot works between the counter and the tables. Seven pizzas are on the counter, three tables have ordered, and two guests must not eat certain things. The robot knows all of this as objects: the classes of `pizza.owl` from last week, plus guests and orders.*

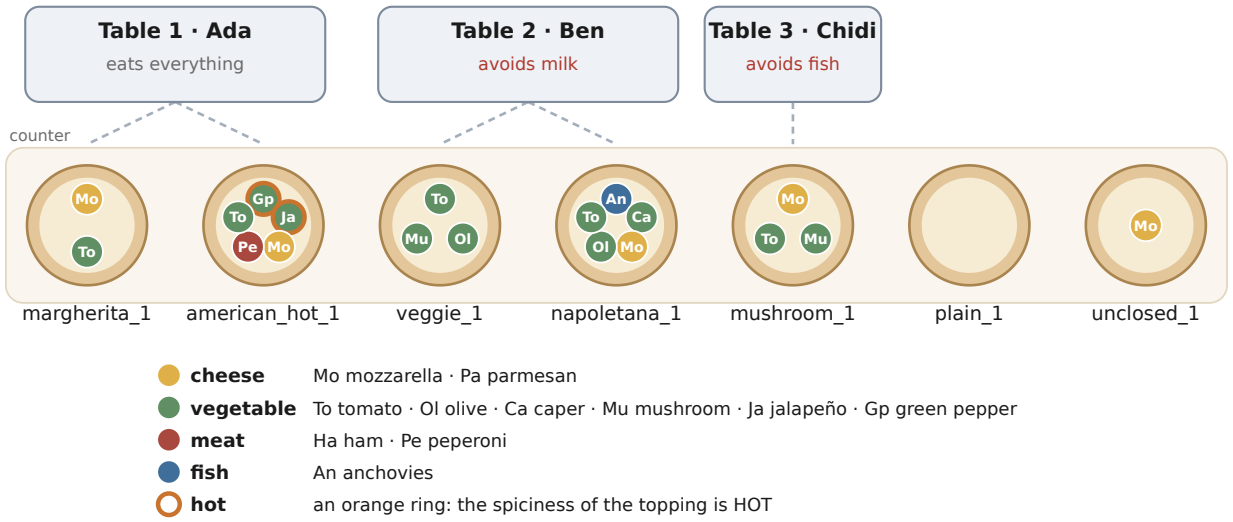



**Figure 2.** What the robot knows at 19:30. Lines connect each table to the pizzas it ordered.

## Warm-up The picture is an interpretation

PAPER

Figure 2 is an *interpretation* of first-order logic. What is its domain? What is the extension of the predicate `CheeseTopping`, and that of the relation `hasTopping`?

**SOLUTION**

The domain is the set of all objects: 7 pizzas, their 19 toppings, 3 guests and 3 orders (the spiciness values and allergens are named by constants). The extension of CheeseTopping is the set of the 5 yellow dots, one mozzarella on each cheesy pizza; that of hasTopping is the set of pairs (pizza, dot on that pizza), 19 pairs. Every query today is evaluated in this *one* interpretation.

REFERENCE · KEEP THIS PAGE OPEN

# EQL, the Entity Query Language

EQL is part of the library KRROOD. Its queries are first-order formulas written over Python objects. The central expression is a **description**: a class and a pattern of attributes.

| EQL | Read it as | First-order logic |
|---|---|---|
| `a(Pizza)` | some object of class Pizza | a variable $x$ with Pizza($x$) |
| `a(Pizza)(toppings=a(CheeseTopping))` | a pizza with a cheese topping | Pizza($x$) ∧ ∃$t$ (hasTopping($x$, $t$) ∧ CheeseTopping($t$)) |
| `a(Pizza)(toppings=[A, B])` | whose topping list is [A, B] | the whole list: exactly these, in this order |
| `a(PizzaTopping)(spiciness=Spiciness.HOT)` | a topping that is hot | spiciness($t$) = hot |
| `a(Pizza)` without `.from_` | … among every pizza the robot knows | the range of $x$ is the registry |
| `.from_(pizzas)` | … among these objects only | the range of $x$ |
| `the(Pizza)(…)` | *the* pizza that … | exactly one object fits |
| `every_is(…)`, `none_is(…)` in place of `a(…)` | every, no topping is … | ∀$t$ (hasTopping → …), ¬∃$t$ (…) |
| `a(PizzaTopping).from_(pizza.toppings)` | a topping of that pizza | a variable $t$ with hasTopping($x$, $t$) |
| `exists(t, φ)`, `for_all(t, φ)`, `HasType(t, C)` | some, every topping …; $t$ is a C | ∃$t$ φ, ∀$t$ φ; C($t$) |
| `not_(φ)`, `or_(φ, ψ)`, `.where(φ, ψ)` | not, or, and | ¬φ, φ ∨ ψ, φ ∧ ψ |
| `set_of(order.guest, pizza)` | return these together | the free variables of the formula |
| `.tolist()` | evaluate now | all assignments that satisfy it |

### Two things to remember

- Writing a query computes nothing: it builds a symbolic expression. Only `.tolist()` evaluates it.
- Every `a(…)` introduces a variable of its own: two `a(Pizza)` are two pizzas, as $x$ and $y$ are two variables. To speak about one pizza twice, name it once, `pizza = a(Pizza).from_(pizzas)`, and use `pizza`.

ACT 1 · 24 MINUTES

# Asking

*“Which pizzas have cheese?”*

> *The chef calls across the kitchen. The cheesy pizzas go back into the oven for a minute, so that the cheese melts. The robot has to find them.*

## (a) A description is a formula

**PAPER** Write `a(Pizza)(toppings=a(CheeseTopping))` as a first-order formula with the free variable *x*, and ring the pizzas of Figure 2 that satisfy it.

**NOTEBOOK** Run 1(a) and compare.

**SOLUTION**

Pizza(*x*) ∧ ∃*t* (hasTopping(*x*, *t*) ∧ CheeseTopping(*t*)). The free variable is what is asked for, the bound variable *t* is a witness. Answers: margherita_1, napoletana_1, american_hot_1, mushroom_1, unclosed_1 (Figure 3). A pattern given for the list `toppings` describes one element: it is the ∃.

The query computes the set of objects that satisfy the formula *in one interpretation*: it checks a model. It does not ask whether the formula follows from a theory, that is, whether it is true in *every* model.

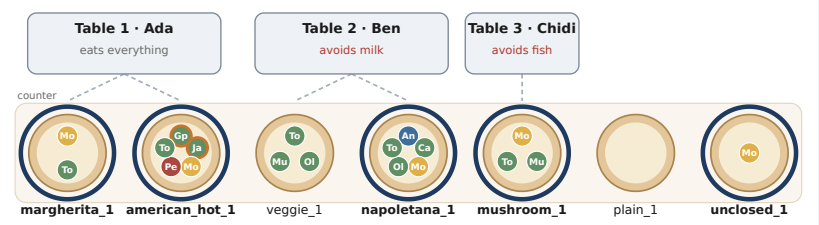



**Figure 3.** The answers of 1(a), ringed.

## (b) Some, only, none

`a(…)` in a pattern says *some topping is* …; `every_is(…)` and `none_is(…)` take its place for *every* and *no* topping:

```
Q1 = a(Pizza)(toppings=every_is(VegetableTopping)).from_(pizzas)
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping), a(TomatoTopping)]).from_(pizzas)
Q3 = a(Pizza)(toppings=none_is(CheeseTopping)).from_(pizzas)
```

**PAPER** Write each query as a formula.

**PREDICT** Mark which pizzas each query returns. Then run 1(b).

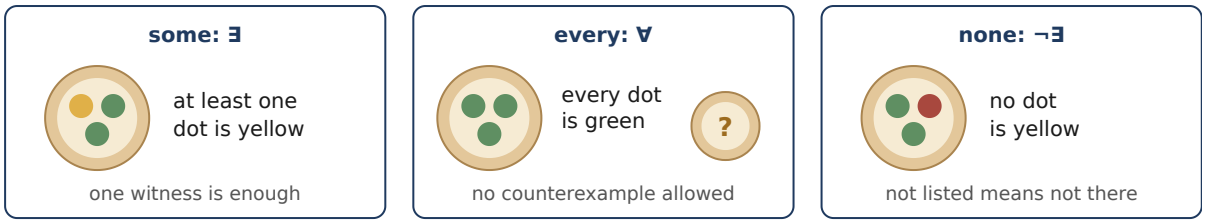



**Figure 4.** The three quantifier patterns, as the robot checks them on one pizza. The small pizza has no toppings at all: is every dot on it green?

### SOLUTION

| | marg. | napol. | am. hot | mush. | veggie | plain | uncl. | formula |
|---|---|---|---|---|---|---|---|---|
| Q1 | | | | | × | × | | Pizza($x$) ∧ ∀$t$ (hasTopping($x$, $t$) → VegetableTopping($t$)) |
| Q2 | × | | | | | | | Pizza($x$) ∧ ∃$t_1$∃$t_2$ (toppings($x$) = [$t_1$, $t_2$] ∧ Mozzarella($t_1$) ∧ Tomato($t_2$)) |
| Q3 | | | | | × | × | | Pizza($x$) ∧ ¬∃$t$ (hasTopping($x$, $t$) ∧ CheeseTopping($t$)) |

- **Q1:** plain_1 has no toppings, so the implication is true for every $t$: there is no counterexample (vacuous truth). It is p3 of Worksheet 2.
- **Q2:** a list of patterns describes the whole list. This implies the closure of Worksheet 2 (some mozzarella, some tomato, and nothing else) and is stronger: it also fixes the number and the order of the toppings. mushroom_1 has a third topping, unclosed_1 only one.
- **Q3:** nobody stated “veggie_1 has no cheese”. The query concludes it because the stored list is taken to be complete: what is not stored is false (closed-world assumption).

### THINK

Last week the OWL reasoner refused to classify `UnclosedPizza` (a pizza with some mozzarella) as vegetarian. In the notebook, write the query `vegetarian` (no meat and no fish topping). Is unclosed_1 an answer? Who is right, the query or the reasoner?

### SOLUTION

```
pizza = a(Pizza).from_(pizzas)
topping = a(PizzaTopping).from_(pizza.toppings)
vegetarian = pizza.where(not_(exists(topping, or_(HasType(topping, MeatTopping),
HasType(topping, FishTopping)))))
```

A pattern names one class per attribute, so *no meat and no fish* names the topping and quantifies over it with `exists`; Q1 is likewise `pizza.where(for_all(topping, HasType(topping, VegetableTopping)))`.

Answers: margherita_1, mushroom_1, veggie_1, plain_1, unclosed_1; in logic $\neg\exists t$ (hasTopping($x$, $t$) $\wedge$ (MeatTopping($t$) $\vee$ FishTopping($t$))).

17 answers, with american_hot_1 and napoletana_1 among them, mean the condition on `topping` has no quantifier, as in `pizza.where(not_(HasType(topping, MeatTopping)), …)`. An unquantified variable means *some*: that query asks for the pizzas with at least one topping that is neither meat nor fish, $\exists t$ (hasTopping($x$, $t$) $\wedge$ $\neg$MeatTopping($t$) $\wedge$ $\neg$FishTopping($t$)), once per such topping; plain_1, with no topping, drops out.

Both are right; they answer different questions. The query evaluates the formula in the one interpretation the robot has, where the list of toppings is complete (closed world). The reasoner asks whether "vegetarian" *follows* from what was said, i.e. holds in every model of it; there are models in which the pizza also has ham (open world). A robot looking at a pizza usually wants the closed-world answer, provided its knowledge of the pizza is complete.

## (c) Following the relations

> *Ada at table 1 does not like spicy food: "Is anything hot on my table?"*

Patterns nest as deep as the objects do: an order has pizzas, a pizza has toppings, a topping has a spiciness.

**PAPER** Write the orders that contain a pizza with a hot topping as a formula with the free variable $o$.

**NOTEBOOK** Write `hot_pizzas` and `hot_orders`, this time without `.from_`: the robot then searches its registry, every object of a `Symbol` class it has created. `hot_orders` may use `hot_pizzas`.

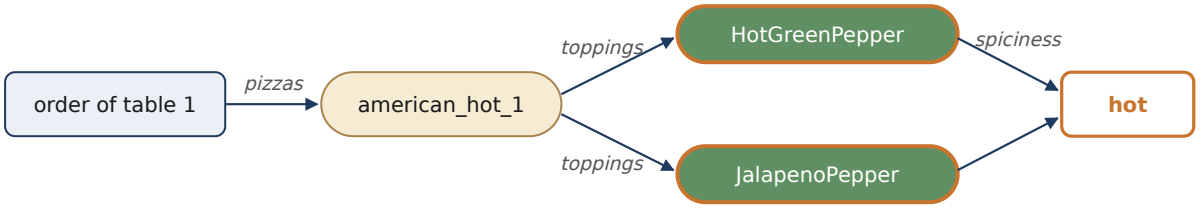



**Figure 5.** The paths from Ada's order to a hot topping. There are two.

**PREDICT** A path such as `pizza.toppings.spiciness` reads *a* topping. Does `pizza.where(not_(pizza.toppings.spiciness == Spiciness.HOT))` return the pizzas without a hot topping? _____

**PREDICT** The notebook then asks for the witnesses themselves: `set_of(order.guest, pizza, topping).where(topping.spiciness == Spiciness.HOT)`. How many answers does it have, while `hot_orders` has one? _____

**SOLUTION**

Order($o$) $\wedge$ $\exists x$ (hasPizza($o$, $x$) $\wedge$ $\exists t$ (hasTopping($x$, $t$) $\wedge$ spiciness($t$) = hot))

```
hot_pizzas = a(Pizza)(toppings=a(PizzaTopping)(spiciness=Spiciness.HOT))
hot_orders = an(Order)(pizzas=hot_pizzas)
```

Answers: american_hot_1, and the order of table 1. The `set_of` query has **two** answers, one per path of Figure 5. An answer is an **assignment** of values to the free variables that makes the formula true. In `hot_orders`, *x* and *t* are bound by ∃: only *o* is free, and the order is returned once, however many witnesses there are.

No, it returns every pizza: `not_` negates the condition on *one* topping, and on plain_1 the condition is false (closed world). *No hot topping* needs ¬∃, as in (b): `toppings=none_is(PizzaTopping)(spiciness=Spiciness.HOT)`.

## (d) *a* or *the*?

> *Ben at table 2 waves: "Bring me the pizza with olives."*

A command does not name an object; it describes one. `the(…)` is still a question, with one more demand: on the number of answers.

**PREDICT** For each command, how many pizzas fit, and what should the robot do? Then run 1(d).

| **0** | **1** | **2 or more** |
|---|---|---|
| objects fit | object fits | objects fit |
| **nothing to refer to** | **refers: bring it** | **ambiguous: ask back** |
| `NoSolutionFound` | `the` answer | `MultipleSolutionFound` |

**Figure 6.** `a(…)` accepts every object that fits; `the(…)` claims that exactly one fits, and raises an exception otherwise.

**SOLUTION**

| Command | Pizzas that fit | What the robot does |
|---|---|---|
| "the pizza with ham" | none | reports that there is no such pizza (NoSolutionFound) |
| "the pizza with fish" | napoletana_1 | brings it |
| "the pizza with olives" | napoletana_1, veggie_1 | asks back (MultipleSolutionFound) |

Russell analysed "the F is G" as $\exists x\,(F(x) \wedge \forall y\,(F(y) \rightarrow y = x) \wedge G(x))$: there is exactly one F, and it is G. If there is no F or more than one, Russell calls the sentence false; Strawson argued that it is neither true nor false, because its presupposition fails. The robot, which refuses instead of answering, takes Strawson's side: picking one pizza at random would be a guess, not obedience.

## (e) Making a description precise

The robot asks back: "The one with fish, or the one without?" Ben: "Without fish."

**PAPER** Write the precise description as a formula.

**NOTEBOOK** Add the condition to `the(Pizza)(toppings=a(OliveTopping))` with `.where(…)`, so that `the` succeeds.

**SOLUTION**

Pizza($x$) ∧ ∃$t$ (hasTopping($x$, $t$) ∧ OliveTopping($t$)) ∧ ¬∃$t'$ (hasTopping($x$, $t'$) ∧ FishTopping($t'$)); tonight only veggie_1 satisfies it.

```
pizza = the(Pizza)(toppings=a(OliveTopping)).from_(pizzas)
topping = a(PizzaTopping).from_(pizza.toppings)
olives_without_fish = pizza.where(not_(exists(topping, HasType(topping,
FishTopping))))
```

Two quantifiers, two variables: the pattern names the olive, `topping` ranges over all toppings. A good question back names a property that one candidate has and the other has not.

Whether a description refers also depends on the **domain**: “the pizza with cheese” does not refer tonight, but “the pizza with cheese of table 3” does (mushroom_1). People narrow the domain by context until “the” works; a robot adds that context to the description, for example `.from_(order_of_table_3.pizzas)`.

ACT 2 · 18 MINUTES

# Defining

*“Which pizzas may carry the green V?”*

> *The chef prints tomorrow's menu, where a green V marks a vegetarian pizza, and a supplier has just delivered two frozen pizzas. In Act 1 you asked which pizzas are vegetarian. Now the word gets a meaning of its own.*

A **definition** gives a description a name. The name is then used in three ways: to *check* one candidate, to *classify* every object, and to *conclude* from the name alone.

## (a) A class declaration says part of what a thing is

A Python object keeps the class it was created with: veggie_1 cannot *become* a vegetarian pizza. A label that an existing object takes on is a **role**, an object of its own that stands for the pizza and reads every attribute through to it:

```
class MenuLabel(Role[Pizza]): ...          # a menu label is a role of a pizza
class VegetarianPizza(MenuLabel): ...
class VeganPizza(VegetarianPizza): ...
```

KRROOD reads the class declarations as statements about every object of a class. Cell 2(a) prints what it reads:

| Read from the declarations | First-order formula |
|---|---|
| every MozzarellaTopping is a CheeseTopping | |
| every Pizza has toppings, each a PizzaTopping | |
| every Order has a guest, which is a Guest | |
| every MenuLabel is a role of one Pizza | |

PAPER Write each line as a formula. Can any of them ever make an object a Pizza, or a VegetarianPizza?

**SOLUTION**

| Read from the declarations | First-order formula |
|---|---|
| every MozzarellaTopping is a CheeseTopping | $\forall x$ (MozzarellaTopping($x$) → CheeseTopping($x$)) |
| every Pizza has toppings, each a PizzaTopping | $\forall x \forall t$ (Pizza($x$) ∧ hasTopping($x$, $t$) → PizzaTopping($t$)) |
| every Order has a guest, which is a Guest | $\forall o$ (Order($o$) → Guest(guest($o$))); guest is a function |
| every MenuLabel is a role of one Pizza | $\forall l$ (MenuLabel($l$) → Pizza(roleTaker($l$))) |

All four are **necessary** conditions: they say what follows *from* being a C, never when something *is* a C. None of them can make an object a pizza. In Python the class is asserted when an object is created, and `HasType` is `isinstance`: it reads the asserted class and never infers one. These are the subclass and range axioms of pizza.owl from last week. Python does not check type hints when the program runs; KRROOD and EQL read them.

## (b) A definition is necessary *and* sufficient

The menu's rule: *a pizza is vegetarian if and only if it has no meat topping and no fish topping.*

**PAPER** Write the rule as one formula. Mark the direction that is sufficient and the direction that is necessary.

**PREDICT** Does the robot accept frozen_1 (mozzarella, ham) and frozen_2 (tomato, capers) for the green V? Then run 2(b). _____

**PREDICT** The notebook then asks which pizzas are vegetarian twice: once of the registry (no `.from_`), once of `.from_(pizzas)`. Do the answers differ? _____

**SOLUTION**

$\forall x$ (VegetarianPizza($x$) ↔ Pizza($x$) ∧ ¬$\exists t$ (hasTopping($x$, $t$) ∧ (MeatTopping($t$) ∨ FishTopping($t$)))). Right to left (←) is sufficient: whatever satisfies the condition is vegetarian. Left to right (→) is necessary.

frozen_1 is rejected (ham), frozen_2 accepted. Checking a candidate is the formula with its variable **bound**, φ($a$): a question whose answer is yes or no. Classifying applies the ← direction as a rule to every pizza, $\forall x$ (φ($x$) → VegetarianPizza($x$)): margherita_1, mushroom_1, veggie_1, plain_1 and unclosed_1 get the role. The role is a Python object of its own, but it stands for the same pizza: the rule adds a fact about something that exists, not a new thing in the kitchen.

The registry also answers frozen_2: creating the frozen pizzas to check them entered them in the registry, but they are not on tonight's menu. The registry is everything the robot has created; `.from_(…)` is the domain the question is about. A check names its domain as `.from_([candidate])`.

**THINK**

A pizza from the other branch arrives with a green V; nobody wrote down its toppings. What may the robot tell Chidi, who avoids fish? And Ben, who avoids milk?

**SOLUTION**

Chidi: it is safe, by the necessary direction: VegetarianPizza($x$) → ¬∃$t$ (hasTopping($x$, $t$) ∧ FishTopping($t$)) follows from the name alone. Ben: nothing, because the definition says nothing about cheese. No query of Act 1 could answer this, since the toppings are unknown. A definition works in both directions: ← to classify, → to conclude from the class.

## (c) Rules with exceptions

The menu gets more labels. The chef's rule of thumb is a rule tree (Figure 7). The declaration `class VeganPizza(VegetarianPizza)` of (a) already says that every vegan pizza is vegetarian.

**PREDICT** Write the label of each pizza in the table, then run 2(c).

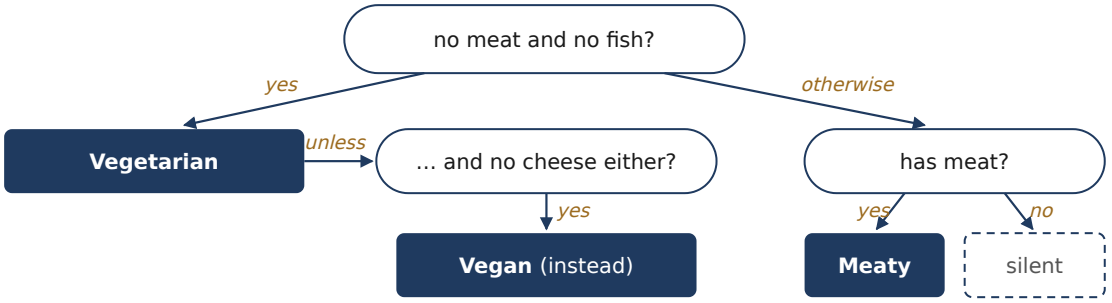



**Figure 7.** A rule tree: a default conclusion, an exception (*unless*, in EQL `refinement`) and an alternative (*otherwise*, in EQL `alternative`).

**SOLUTION**

| margherita | napoletana | american hot | mushroom | veggie | plain | unclosed |
|---|---|---|---|---|---|---|
| Vegetarian | **none** | Meaty | Vegetarian | Vegan | Vegan | Vegetarian |

napoletana_1 has fish, so the first condition fails; it has no meat, so the alternative fails as well. The rules are **silent**, and silence is not "no": they never concluded "not vegetarian"; reading it so would be one more closed-world step. The *unless* branch is an exception: a more specific conclusion replaces the general one. Rule trees of this shape are known as ripple-down rules.

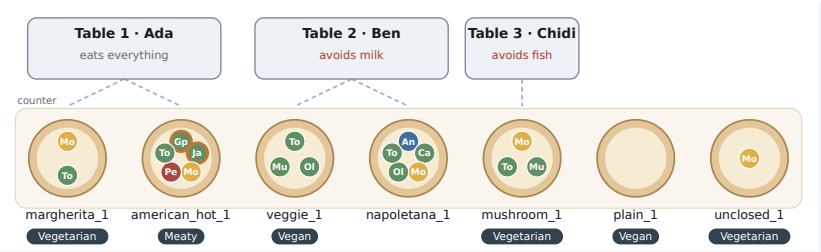



**Figure 8.** The menu labels, as derived by the rule tree.

ACT 3 · 12 MINUTES

# Constraining

*“Ben must never get milk.”*

> *Ben avoids milk. Nowhere does it say “napoletana_1 contains milk”: it has to be computed. Mozzarella is a cheese, cheese contains milk, and the pizza has that topping.*

## (a) Stored or computed?

`symbolic_function` turns a Python function into a symbol of the query language, and a `Predicate` is a relation decided by a computation.

**PAPER** For each fact below, is it stored in the objects (S) or computed (C)?

**NOTEBOOK** Complete `allergens` and `IsSafeFor`.

**SOLUTION**

| **hasTopping(*x*, *t*)** | **guest(*o*) = *g*** | **∀*t* (Mozzarella(*t*) → Cheese(*t*))** | **milk ∈ allergens(*x*)** | **IsSafeFor(*x*, *g*)** |
|---|---|---|---|---|
| S: the list `toppings` | S: the field `guest` | S: the class hierarchy | C: `allergens` | C: the predicate |

```
@symbolic_function
def allergens(pizza):
    found = {Allergen.GLUTEN}
    if any(isinstance(t, CheeseTopping) for t in pizza.toppings):
found.add(Allergen.MILK)
    if any(isinstance(t, FishTopping) for t in pizza.toppings):
found.add(Allergen.FISH)
    return found

class IsSafeFor(Predicate):
    def __call__(self): return not (allergens(self.pizza) & self.guest.avoids)

pizza = a(Pizza).from_(pizzas)
safe_for_ben = pizza.where(IsSafeFor(pizza, ben))       # veggie_1, plain_1
```

In a query, stored and computed relations look the same. Giving a symbol a program as its meaning is called **procedural attachment**.

## (b) A constraint is a query that must have no answer

The kitchen's rule: every pizza of an order is safe for the guest of that order.

**PAPER** Write the rule as a formula (hasPizza, guest, IsSafeFor), and then its negation, pushed inwards.

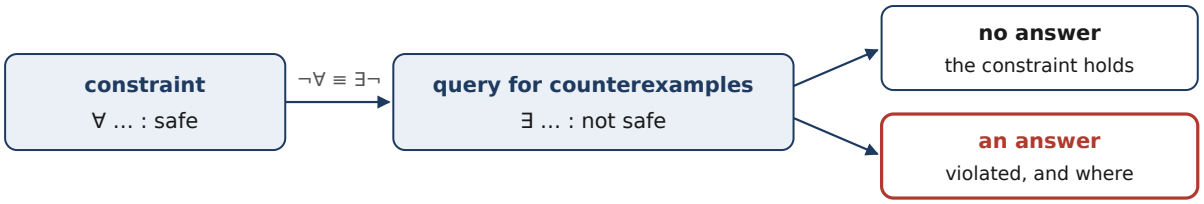



**Figure 9.** A universal statement is violated exactly when a counterexample exists.

**PREDICT** How many violations are there tonight? Draw each one as a line in Figure 2.

**NOTEBOOK** Write `dangerous` and check.

### SOLUTION

Constraint: $\forall o\ \forall x$ (hasPizza($o$, $x$) → IsSafeFor($x$, guest($o$))).
Negation: $\exists o\ \exists x$ (hasPizza($o$, $x$) ∧ ¬IsSafeFor($x$, guest($o$))). As a query, $o$ and $x$ are left free, so that each answer names a culprit:

```
order = an(Order).from_(orders)
pizza = a(Pizza).from_(order.pizzas)
dangerous = set_of(order, pizza).where(not_(IsSafeFor(pizza, order.guest)))
```

One violation: table 2, Ben, napoletana_1 (mozzarella, hence milk). EQL could evaluate the ∀ directly, but the answer would only be “false”; the counterexample query says which plate not to serve.

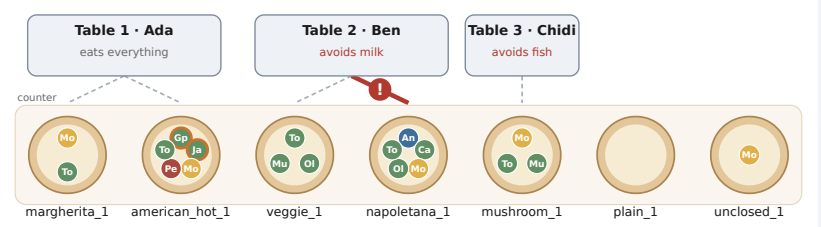



**Figure 10.** The one violation of the constraint.

### THINK

Suppose the kitchen's rule were an axiom of last week's ontology. What would the reasoner report for tonight's facts? And if the mozzarella of the Napoletana had simply not been recorded?

### SOLUTION

An axiom does not check facts; it restricts the models. Facts that contradict it leave no model at all: the ontology is **inconsistent**, and from an inconsistent theory every formula follows. The reasoner reports the inconsistency, not a plate. If the mozzarella had not been recorded, there would be no contradiction: in the open world, the reasoner would rather conclude that the Napoletana is safe for Ben, because the axiom demands it. Used as an **integrity constraint**, the same formula is checked against the one interpretation; the knowledge base stays usable, and the answer is the culprit.

ACT 4 · 12 MINUTES

# Deriving

*The kitchen writes allergen warnings.*

> *The menu must list the allergens of every pizza. Instead of computing them for every question, the kitchen writes them down once: a slip of paper on each pizza.*

The labels of Act 2 were facts about pizzas that exist. A warning is different: it is a *new* object.

## (a) A rule creates objects

`inference(AllergenWarning)(pizza=D, allergen=Allergen.MILK)` creates one new warning object for every pizza that fits the description D. With D the description of Act 1(a), it is the rule

```
∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
```

PAPER Which part of the formula is D, and which part does the rule build?

PREDICT With a fish rule and a gluten rule for every pizza added, how many warnings are there?

---

NOTEBOOK Write both rules.

**SOLUTION**

The left side of → is the **body** of the rule: exactly the description of 1(a). The right side is the **head**: for every answer of the body, a new object *w* is created as the witness of ∃*w*.

```
fishy = a(Pizza)(toppings=a(FishTopping)).from_(pizzas)
fish_warnings   = inference(AllergenWarning)(pizza=fishy,
allergen=Allergen.FISH).tolist()
gluten_warnings = inference(AllergenWarning)(pizza=a(Pizza).from_(pizzas),
allergen=Allergen.GLUTEN).tolist()
```

13 warnings: 5 milk, 1 fish, 7 gluten. Deriving all conclusions in advance is **forward chaining** (materialisation). The predicate of Act 3 works the other way: it derives its conclusion only when a question needs it, which is the strategy of **backward chaining**.

Compare the label rule of Act 2, $\forall x\,(\varphi(x) \rightarrow \text{VegetarianPizza}(x))$: its head has no ∃, so it only adds facts about the objects there are. Rules of that kind never create objects; rules with ∃ in the head do, one witness for every answer of the body.

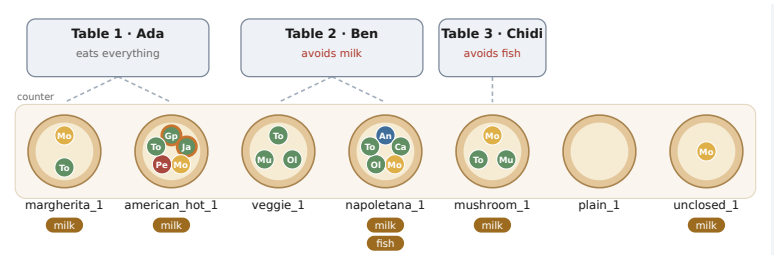



**Figure 11.** The milk and fish warnings (gluten, on every pizza, is left out).

## (b) The world moves on

> *It is busy. The cook mixes up two tickets and grates parmesan over veggie_1, Ben's pizza.*

**PREDICT** What do the three sources below say about veggie_1 now? Which of them protects Ben? Then run 4(b).

**SOLUTION**

| 1. the constraint of Act 3, checked now | 2. the milk warnings derived in (a) | 3. the label of veggie_1 (Act 2), derived again |
|---|---|---|
| two violations: napoletana_1 *and* veggie_1 | no warning for veggie_1 | Vegetarian (it was Vegan) |

- Only the constraint protects Ben. It was written before the mistake and is evaluated against the world as it is *now*.
- The stored warnings were derived at a fixed moment and are stale until the rule runs again. Keeping derived facts in line with the facts they were derived from is called **truth maintenance**; a robot, whose world changes all the time, needs it constantly.
- "Vegan" was withdrawn because a fact was *added*. Reasoning in which more knowledge can remove a conclusion is **non-monotonic**; here it is so because the vegan rule relies on the absence of cheese. Classical first-order logic is monotonic: what follows from a set of formulas still follows when formulas are added.

ACT 5 · 7 MINUTES

# Generating

*“A pizza with mozzarella and tomato, please.”*

> *Dana sits down at table 4 and orders. Her order is a description, exactly like those of Act 1. What is new is who answers it.*

A query in EQL is a symbolic object; nothing is computed when it is written. A **backend** is whatever answers it, and the same description can be handed to different backends:

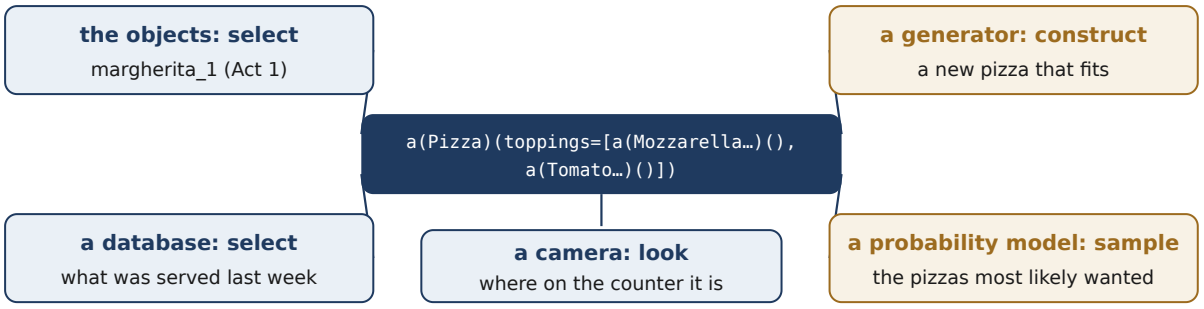



**Figure 12.** The description is written once, symbolically. Selecting (blue) and constructing or sampling (ochre) are different ways of answering it.

## (a) Select or construct?

**PREDICT** What does the counter answer, and what does the kitchen answer? Dana adds “… and a green pepper, I don't mind how hot” (`spiciness=...`, left open). How many pizzas does the kitchen offer? _____

**NOTEBOOK** Run Act 5.

**PAPER** Every pizza the kitchen offered is now in the registry. Which question of tonight would get a different answer if the robot asked it again? _____

**THINK**

In Act 1, every answer was an object of the interpretation in Figure 2. What is an answer here? What does it mean, in terms of models, that Dana's open description has several answers?

**SOLUTION**

The counter (a selecting backend) answers margherita_1; the kitchen (a generative backend) answers a *new* pizza with exactly these toppings. The open description has three answers, one per value of `Spiciness`.

An answer is now a new object that, added to the interpretation, makes the description true: answering means *constructing* a model instead of checking one. An underspecified description can be made true in several ways; the generator enumerates them, a probabilistic backend would rank them by how likely they are.

Every evaluation reads the registry as it is then. Asked again, `hot_pizzas` of 1(c) and the vegetarian question put to the registry in 2(b) also find the kitchen's offers, pizzas nobody ordered. Every question with `.from_(pizzas)` or `.from_(orders)` keeps its answer. Ask the registry when the question is about everything the robot knows; give a domain when it is about a particular set: tonight's counter, one candidate.

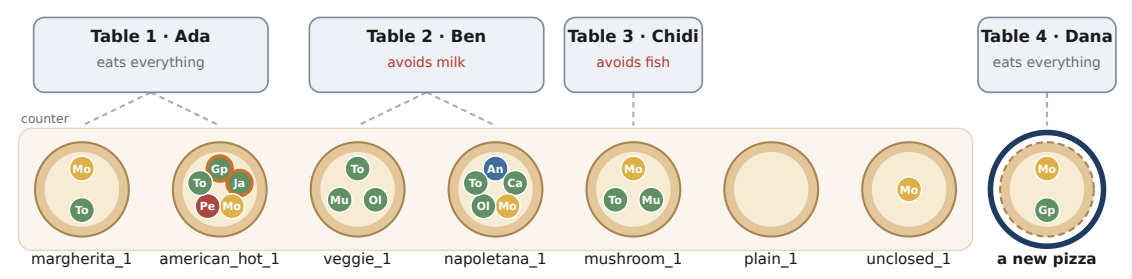



**Figure 13.** The world has grown: a pizza that did not exist at 19:30.

## (b) One plan, many answerers

The robot's next task is a plan with open slots, each written as a description:

**Bring** ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

**PAPER** Which kind of backend should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probability model, **G** a generator.

**SOLUTION**

| ① safe pizza | ② its pose | ③ Ben's table | ④ grasp | ⑤ have one made |
|---|---|---|---|---|
| R and K: IsSafeFor over the pizzas | L: the knowledge base knows *that* it is there, not exactly *where* | K: the orders | S: sampled, then tried | G: Act 5(a) |

Several answers are defensible; what matters is that the plan does not say *who* answers. It states what it needs, symbolically, and each slot goes to the backend that can answer that kind of description. The robots of our current research work this way: in a sorting task, the piece to pick up is *looked for* by perception, where it stands is *read off* the robot's digital twin, the hole it fits through is *concluded* by rules, and how to grasp it is *sampled*: one plan, four backends.

EPILOGUE · 5 MINUTES

# The whole picture

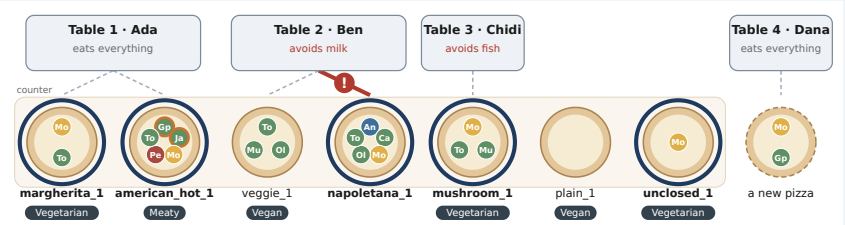



**Figure 14.** What the robot knows at 20:00: the answers to the chef's question (ringed), the menu labels, the violated constraint, and Dana's new pizza.

## (c) Your summary

PAPER For each use of a description, give tonight's example and what it is in logic.

**SOLUTION**

| Use | The question behind it | Tonight's example | In logic |
|---|---|---|---|
| 1 Ask | Which objects fit? | the cheesy pizzas; who gets which hot topping; *the* pizza with olives and without fish | the assignments to the free variables that satisfy a formula in one interpretation; *the*: exactly one |
| 2 Define | What is a C? | the green V: frozen_2 accepted, five pizzas classified; the menu labels | C($x$) ↔ φ($x$): ← classifies (a rule without ∃ in its head), → concludes from the class; class declarations give necessary conditions only |
| 3 Constrain | Does anything violate it? | no pizza unsafe for its guest | a ∀ checked through its counterexamples ∃¬; an integrity constraint, not an axiom |
| 4 Derive | What new object does each fit get? | the allergen warnings | a rule with ∃ in its head: one new witness per answer of the body, by forward chaining |
| 5 Generate | What could fit? | Dana's pizza; the slots of the plan | constructing new objects that make the description true, chosen among many |

Two questions tell the uses apart. Does the use **read** the knowledge base (1, 3) or **write** to it (2, 4, 5)? And must the answer fit the world, as for a question, or must the world fit the formula, as for a constraint and a specification? Deriving and generating both create objects: a rule's objects are *forced*, one per answer of its body; a generator *chooses* one of many that fit, so it has to search.

> The non-monotonic rule tree belongs to defining: its conclusions are labels of existing pizzas, and an added fact (the parmesan of 4(b)) withdrew one of them.

### Take home

One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a definition, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.